

Introduction:

The E-Sports Team here on campus is having trouble keeping an organized schedule. So we developed a program that will take in an ICS link from an Outlook calendar, which is our preferred program for our school. Then we parse this ICS file using iCalendar. Finally, we used PyQt6 for a graphical interface that displays a calendar from the ICS link, with the current day on the left and the next 6 days following. There is also a status screen at the bottom of the graphical interface that takes in a word or phrase from the calendar feed, and will change color depending on what the availability is set for that current event.

Time Log:

Kyle Becker

Time Spent	Tasks	Accomplishments
1 hour	Researching frameworks	Started to find some frameworks we can use
2 hours	Implementing the front end	I started to implement the bare bones of the front end using a PyQt6 GUI
3 hours	Getting the ics feed to show up on the front end	I got a barebones ICS feed to show up that currently doesn't work with recurring events
3 hours	Improving the front end and making it nicer to look at	I improved the front end and added bubbles behind the events, and made it so the events show up in chronological order
3 hours	Make it so the calendar automatically updates and is reordered so the current day is on the left and the next 6 days follow	Implemented the automatic updating and reordered the layout
3 hours	Finish implementing recurring events into the front end	I managed to finish implementing the front end to work with the recurring

Parker Reece

1 hour	Researching how to make a parser	Found iCalendar to use for parsing
--------	----------------------------------	------------------------------------

3.5 hours	Read through the iCalendar docs and worked on getting a basic parser implementation working	Got a basic parser started.
4 hours	Expanded the parser	I got the parser working on everything except for recurring events
4 hours	Worked through test driven development for recurring events	Finished the parser including recurring events

Requirements:

- A graphical interface that displays a calendar from an ICS link
- A status bar that changes depending on what the event is, for example, being available, available but be quiet, and booked.
- A separate screen with an input to take in an ICS link that will then download the file and store it locally

Design:

We chose to use Python to code this app in. We made this decision because it is the language that we both know the best. We used several main libraries for our app. The first is a library called iCalendar. It is a library for parsing ICS files and extracting the data from them. This library made building the parser much easier, as we didn't have to deal with the raw data from the ICS file. Instead we mostly interacted with the calendar object that we extracted data from to create a list of event objects. For the front end of the system we used a library called PyQt6. It is used to create the GUI and to draw the calendar on the screen. PyQt6 is based off of the Qt6 application framework, it allows Python developers to develop a cross-platform desktop applications with a native looking GUI.

Our code is structured to be in several files. The main file that connects everything and has the main loop is found in `room_availability_app.py`. We then have several other main ideas broken out. The `Parsing.py` file contains all of the code for the parsing back end. The configuration manager file is the main way that the app can be configured, including storing the ICS url. The setup page defines a class that runs when the app starts that allows the user to configure things, in concert with the config manager class. One design pattern that we used is a facade. The parser has one entry point. It has a function, `parse_from_calendar()`, that takes in the url, a start date, and an end date and returns a list of events in that window. This is the one point where the parsing system interacts with the rest of the system and the rest of the complexity of the parser is hidden through abstraction with this one function.

Implementation:

The Room Availability Dashboard is a PyQt6 desktop application that displays a weekly view of room bookings pulled from a publicly hosted ICS calendar file. The system is structured

in three layers: a data layer (Parsing.py) that fetches and parses the ICS file into internal Event objects, a configuration layer (config_manager.py) that persists the calendar URL and a local cache to disk, and a presentation layer (setup_page.py and room_availability_app.py) that handles all user interaction and rendering. On first launch, a setup page collects the user's ICS URL. On subsequent launches, the app loads from the saved configuration and displays the current 7-day window of events, color-coded by room availability status. A background timer refreshes the calendar data every three hours without requiring user action.

First-time Setup:

The first functional requirement for the program was that the first time you opened the app, you are to be guided through providing the ICS link before the calendar is shown. This implementation used PyGt6's signal and slot mechanism, which uses two separate windows to communicate without being tightly coupled together.

The SetupPage class defines a custom signal:

```
class SetupPage(QWidget):
    setup_completed = pyqtSignal()
```

When the user successfully submits a valid URL, the save_setup method fetches the ICS file, saves it locally, stores the URL in the configuration, and then emits the signal:

```
def save_setup(self):
    url = self.url_input.text().strip()

    if not url:
        QMessageBox.warning(self, "Empty Link", "Please enter an ICS calendar link.")
        return

    try:
        ics_data = fetch_ics_from_url(url)
        self.config_manager.save_ics_file(ics_data)
        self.config_manager.set_ics_url(url)
        self.setup_completed.emit()

    except Exception as e:
        print(f"Error in save_setup: {e}")
        import traceback
        traceback.print_exc()
        QMessageBox.critical(
            self,
            "Error",
            f"Failed to save calendar:\n{str(e)}"
        )
```

In the main application, the entry point is connected to a slot that closes the setup page and opens the main window:

```
if not config_manager.has_ics_configured():
    setup_page = SetupPage()

    def on_setup_complete():
        if setup_page:
            setup_page.close()
            create_and_show_main_window()

    setup_page.setup_completed.connect(on_setup_complete)
    setup_page.show()
else:
    create_and_show_main_window():
```

Designing the start page this way means the SetupPage never needs to know that RoomAvailabilityApp exists. It simply sends a signal between the two parts that the job is done.

Rendering Events into Day Columns:

The main part of this program was to show each day of the week in 7 columns, 1 for each day of the week, and in each column, there will be a list of events for that day. This is handled by the `get_events_for_day`, which filters the event list down to those matching a given date, and the rendering loop in `__init__` that builds the UI columns from those results. It also converts each event's start time to the local timezone before comparing dates because the ICS file stores times in UTC:

```
def get_events_for_day(self, date):
    """Get events for a specific date."""
    day_events = []
    local_tz = datetime.now().astimezone().tzinfo
    target_date = date.date()

    for event in self.events:
        if event.all_day:
            event_date = event.start.date()
        else:
            event_date = event.start.astimezone(local_tz).date()

        if event_date == target_date:
            day_events.append(event)

    day_events.sort(key=lambda e: e.start)
```

```

formatted_events = []
for event in day_events:
    if event.all_day:
        formatted_events.append(f"{event.title} (All Day)")
    else:
        start_time = event.start.astimezone(local_tz).strftime("%I:%M %p")
        end_time = (event.end.astimezone(local_tz).strftime("%I:%M %p")
                     if event.end else "")
        if end_time:
            formatted_events.append(f"{start_time} - {end_time}: {event.title}")
        else:
            formatted_events.append(f"{start_time}: {event.title}")

return formatted_events

```

The rendering loop then calls this method for each of the 7 days and builds a QFrame column for each one. If an event exists, each is styled with a blue appearance. If there are no events, a label saying “No events” is shown instead:

```

for i in range(7):
    current_date = today + timedelta(days=i)
    day_name = current_date.strftime('%A') # Full day name (Monday, Tuesday, etc.)
    day_label = f"{day_name}\n{current_date.strftime('%m/%d')}"

    zone = QFrame()
    zone setFrameStyle(QFrame.Shape.Box)
    zone.setLineWidth(2)
    zone.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
    zone_layout = QVBoxLayout(zone)
    zone_layout.setContentsMargins(6, 6, 6, 6)
    zone_layout.setSpacing(4)
    zone_layout.addWidget(QLabel(f"<b>{day_label}</b>"))

    day_events = self.get_events_for_day(current_date)
    if day_events:
        for event_text in day_events:
            event_label = QLabel(f"{event_text}")
            event_label.setWordWrap(True)
            event_label.setStyleSheet(
                "background-color: #0078d4;"
                "color: white;"
                "border: 1px solid #005a9e;"
                "border-radius: 10px;"
                "padding: 6px 8px;"
            )

```

```

        "margin-bottom: 4px;"
    )
    zone_layout.addWidget(event_label)
else:
    no_events_label = QLabel("No events")
    no_events_label.setStyleSheet("color: gray; font-style: italic;")
    zone_layout.addWidget(no_events_label)

days_layout.addWidget(zone)
self.day_zones.append(zone)

```

The 7 columns are placed inside a QScrollArea with horizontal scrolling, so that all days remain visible.

The parsing app takes in a URL and gets the ICS file in the form of raw bytes. These are then passed into the main parse function. It then uses the icalendar library to create a Calendar object.

```

calendar = Calendar.from_ical(ics_data)
events: List[Event] = []

```

It then loops over the events in this calendar object so that we can create a list of custom event objects that the rest of the system can interact with. The function starts by getting basic information like the title and start time.

for component in calendar.walk():

```

    if component.name != "VEVENT":
        Continue

    uid = str(component.get("UID"))
    title = str(component.get("SUMMARY", ""))
    location = str(component.get("LOCATION", ""))
    description = str(component.get("DESCRIPTION", ""))

    dtstart_raw = component.get("DTSTART").dt
    start, all_day = normalize_datetime(dtstart_raw)

    dtend_component = component.get("DTEND")
    if dtend_component:
        end, _ = normalize_datetime(dtend_component.dt)
    else:

```

```
end = None
```

Next the app gets the recurrence information and just stores it for later. A later function uses this information to create the necessary events that are passed into the front end. It also gets information about exceptions to the recurrence (Ex: repeats every thursday except the 27th)

```
rrule_component = component.get("RRULE")
# Store as raw RFC 5545 string so dateutil.rrulestr gets proper formatting
# (handles BYDAY weekday names, UNTIL timestamps, WKST, etc.)
rrule = rrule_component.to_ical().decode("utf-8") if rrule_component else None
```

```
exdates_list: List[datetime] = []
exdate_component = component.get("EXDATE")
```

```
if exdate_component:
    if not isinstance(exdate_component, list):
        exdate_component = [exdate_component]
```

```
for ex in exdate_component:
    for dt in ex.dts:
        ex_dt, _ = normalize_datetime(dt.dt)
        exdates_list.append(ex_dt)
```

Last it finds whether or not the room is reserved. This information is later used to decide if the indicator should be yellow or red.

```
if title.startswith("Reserved"):
    exclusive = True
else:
    exclusive = False
```

Finally it creates the Event object and appends it to the list. After it has looped through all of VEVENT objects, it returns the list of Event objects.

```
event = Event(
    uid=uid,
    title=title,
    start=start,
    end=end,
    location=location,
    description=description,
    rrule=rrule,
    exdates=exdates_list,
```

```
        all_day=all_day,  
        exclusive = exclusive  
    )  
  
    events.append(event)  
  
    return events
```

Test Procedure:

One test that we did was the test driven development of recurring event handling. Specifically it was the creation of the `expand_event()` function. We had a total of eleven automated tests that checked various cases for recurring event handling. These automated tests can be found in the `parser_test.py` file. They use the `pytest` library. It took approximately five iterations to get all of the tests passing.

Another test was the manual test of the full parsing system. This test was designed to make sure that the parsing system was working from start to finish. It involved using the `parse_from_calendar()` to parse an ICS file from a URL and printing out the list of events it returns to make sure that it correctly captured all of them.

For the first manual test for the frontend, you test and see if you get a error when you try and save and start the calendar without inputting a ICS link.

For the second manual test related to the front end, it relates to the button to reset the calendar data. You click remove calendar in the main calendar window, and see if the window resets and the program reopens to the start page.

The final test that we did was a manual test for the whole system. This test was to run the main app file and input an ICS URL. We then manually checked to see if the correct events were placed in the correct locations on the GUI.

References:

iCalendar documentation: <https://icalendar.readthedocs.io/en/stable/index.html>

PyQt6 documentation: <https://pythonguides.com/pyqt6/>

Github Copilot

Claude Code